Remembering a Ball You Cannot See: Shortcuts, Memory and Attention in Occluded Pong

Giulio Sighieri
Bocconi University, Deep Learning for Computer Vision

Abstract

Pong is easy for reinforcement-learning agents to win, but often for the wrong reason: they read the ball's position off the opponent's paddle, which follows the ball almost perfectly. We make the shortcut and the ball harder to use. A fog hides the ball in the agent's half of the court, and a mask hides the opponent's paddle. We then ask whether agents learn to remember a ball they can no longer see, and whether they use their architecture's tools to do it. We train three agents with PPO: a CNN on four stacked frames, a CNN with an LSTM memory, and a CNN whose LSTM reads the frame through memory-steered spatial attention. We then open them with linear probes, memory wipes and attention blocking. Under fog with the opponent's paddle visible, every agent still plays through that paddle and loses every point without it. Only when both are hidden do the agents remember the ball. The LSTM's memory then holds the hidden ball's height, and wiping it as the ball disappears cuts the share of balls returned from 98% to 30%. The attention agent remembers only what its attention let in: blocking its attention to the ball just before the fog erases the ball from its memory and makes it miss. The memoryless CNN finds another way. It moves early and lets its own paddle carry the information through the fog. The attention agent rarely learns the hardest view from scratch (1 of 3 seeds), but always learns it after pre-training on the clean game.

1 Introduction

Pong was among the first Atari games solved from pixels by deep reinforcement learning [12, 13], and it remains the usual first test for a new agent. Winning it, however, does not mean the agent has understood it. Delfosse et al. [6] found that agents trained on Pong track the opponent's paddle rather than the ball. The built-in opponent follows the ball closely, so its height is a good proxy for the ball's. HackAtari [5] confirmed the diagnosis: when the opponent stops moving, the agents fall apart. This is a case of shortcut learning [7]. A cue that works on the training distribution replaces the one the task is about.

We use this to build a controlled test of memory. A *fog* hides the ball in the agent's half of the court, so the ball vanishes on its way towards the agent and reappears only at its paddle. A *mask* hides the opponent's paddle. With both, the agent cannot see the ball when it matters, nor anything that tracks it. To return the ball it must carry the ball's position and velocity through the fog itself. The question is simple: *can agents learn to remember a ball they can no longer see, and do they use the tools in their architecture to do it?*

We compare three agents that differ only in how they carry information over time:

- a **CNN** on four stacked frames, the DQN setup, with no memory beyond the stack;
- a **CNN + LSTM**, which sees one frame per step and keeps a recurrent memory [8];
- a **CNN + attention + LSTM**, a simplified version of Mott et al. [14], whose memory decides where on the frame its attention looks.

We train each on all four combinations of fog and mask and then look inside them with causal tests.

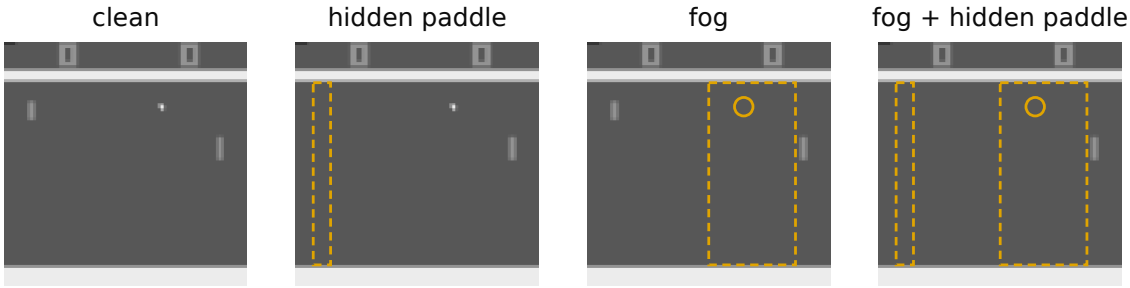


Figure 1: The four views of the same moment of a game. The fog blanks the agent's half of the court (columns 42–71 of the 84×84 frame); the mask blanks the opponent's paddle (columns 6–11). Dashed boxes mark the hidden regions; the circle marks the ball where it is hidden. The agent (right) always sees its own paddle.

Findings.

1. The shortcut survives the fog. With only the ball hidden, every agent plays through the opponent's paddle: erasing or mirroring that paddle costs it every point, and moving it by 4 px most of them.

2. With the shortcut gone, the memory agents remember the ball. A linear probe reads the hidden ball's height from the LSTM's state (R^2 0.87–0.97, against 0.27–0.58 for an untrained network). Wiping that state as the ball disappears cuts the share of balls returned from 0.98 to 0.30.

3. The attention agent remembers what it looked at. Blocking its attention to the ball just before the fog leaves the ball out of its memory and makes it miss most balls in 3 of 4 models. The same block elsewhere changes nothing.

4. The memoryless CNN commits early. It moves its paddle while the ball is still visible and holds it there, so its own paddle carries the information through the fog.

5. The attention agent fails the hardest view from scratch in 2 of 3 seeds, but pre-training on the clean game for 10M steps and fine-tuning for 5M (the same 15M total) makes all three seeds win.

2 Setup

Environment. We use ALE Pong [3] through Gymnasium with the standard preprocessing [11]: 84×84 grayscale frames, frame skip 4, up to 30 no-op starts, and no sticky actions. The occlusions are applied to the observation only, so the game itself is unchanged. Figure 1 shows the four views: **clean**, **hidden paddle** (mask), **fog**, and **fog + hidden paddle**. Under fog, an incoming ball is invisible for about 14 agent steps. For analysis, every frame's object positions are read from the unoccluded frame, which the agent never sees.

Agents. All three agents share the Nature-CNN convolutions [13] and the actor and critic heads (Table 1).

- **CNN** sees four stacked frames and maps them to 512 features.
- **CNN + LSTM** sees one frame per step; its 512 features feed an LSTM with 128 units [9].

agent	input per step	memory	parameters
CNN	4 stacked frames	none	1.69M
CNN + LSTM	1 frame	LSTM, 128 units	2.01M
CNN + attention + LSTM	1 frame	LSTM, 128 units; 2 attention heads	0.23M

Table 1: The three agents. All use the Nature-CNN convolutions and the same PPO hyperparameters.

- **CNN + attention + LSTM** also sees one frame. Its convolutions produce a 7×7 map of 64 features, to which two fixed channels add each cell's x and y position. Each of two heads scores the 49 cells with a query computed from the LSTM's previous hidden state h_{t-1}. The softmax-weighted sum of the cells' values (the readout) is the LSTM's only input. What the agent remembers therefore decides what it looks at next. We layer-normalise queries and keys, without a learned scale; without this the attention stays uniform at initialisation and never learns where to look. It has 7–9 times fewer parameters than the other two.

Training. PPO [15] follows CleanRL [10]. It uses 16 parallel games, 256 steps per game per update, 4 epochs of 4 minibatches, $\gamma = 0.99$, GAE $\lambda = 0.95$, clip 0.1, entropy coefficient 0.02 and Adam with learning rate 10^{-3} annealed linearly to zero. The recurrent agents are trained on whole 256-step sequences, with the memory reset at the end of each game. Every agent is trained from scratch on every view for 15M agent steps, with three seeds. The *curriculum* variant starts from the clean model's 10M-step checkpoint and is fine-tuned on the target view for 5M steps, with a fresh optimiser and the learning rate restarted and annealed over those 5M steps. Its total, 15M steps, equals the scratch budget.

Evaluation. Each frozen model plays 30 games against the built-in opponent, with environment seeds 10000–10029 and its stochastic policy. A game's return is points won minus points lost, from -21 to $+21$. The causal tests replay each ball flying towards the agent from an exact emulator snapshot. They then compare the share of these balls the agent returns under an untouched branch and under an intervention. A second untouched branch, with different action noise, agrees with the first within 0.04 everywhere.

3 Results

3.1 Fog plus hidden paddle separates the agents

Figure 2 shows how well each agent plays its own view.

- **Every agent learns the clean, hidden-paddle and fog views.** The one exception is the attention agent from scratch, which fails one seed on the hidden-paddle and fog views.
- **Fog + hidden paddle separates them.** The LSTM agent wins with every seed (+20.2 on average). Surprisingly, the memoryless CNN wins with two of three seeds (+19.8, +19.1, +1.6). The attention agent wins with only one (-7.0 on average).
- **The curriculum rescues the attention agent.** Fine-tuned from its clean model, it wins with all three seeds (+19.5 to +20.6) within about 3M steps.

The same short fine-tuning is not enough for the other two agents. With 5M steps, the CNN and LSTM fine-tunes are still losing on fog + hidden paddle (-15.5 and -8.9). This is not negative transfer: with the scratch budget and schedule (15M fine-tuning steps), every CNN and LSTM fine-tune wins. Two of the LSTM fine-tunes even learn faster than scratch (+9.1 and +13.8 after 2M steps, against -8.7 and -8.2). The CNN from scratch needs 7.5–10M steps before it

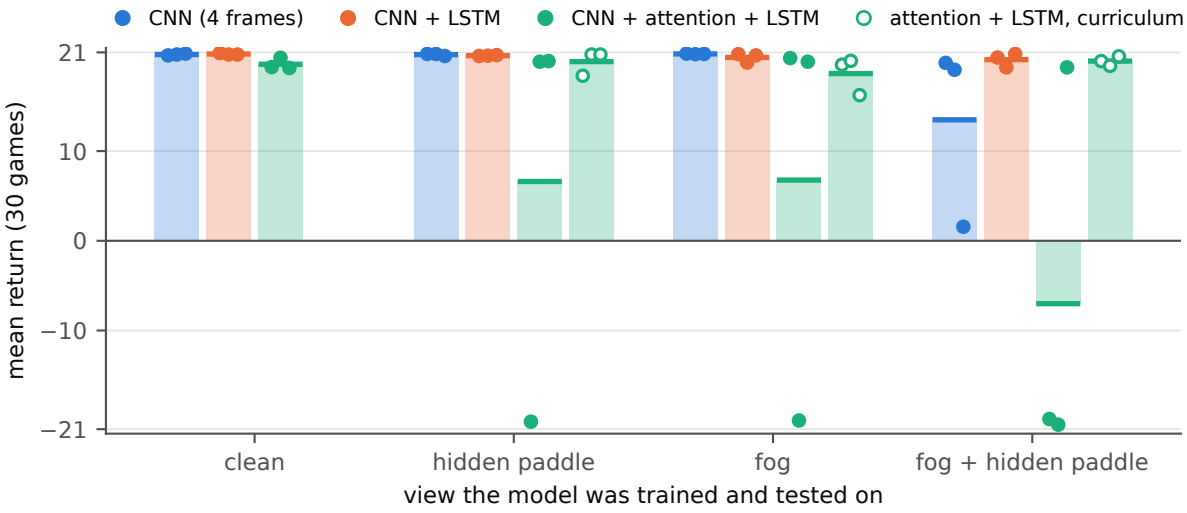


Figure 2: Mean return of each final model on the view it was trained on (bars: mean over seeds; dots: seeds; 30 games each). The attention agent's curriculum runs (hollow) are pre-trained on the clean view for 10M steps and fine-tuned on the target view for 5M.

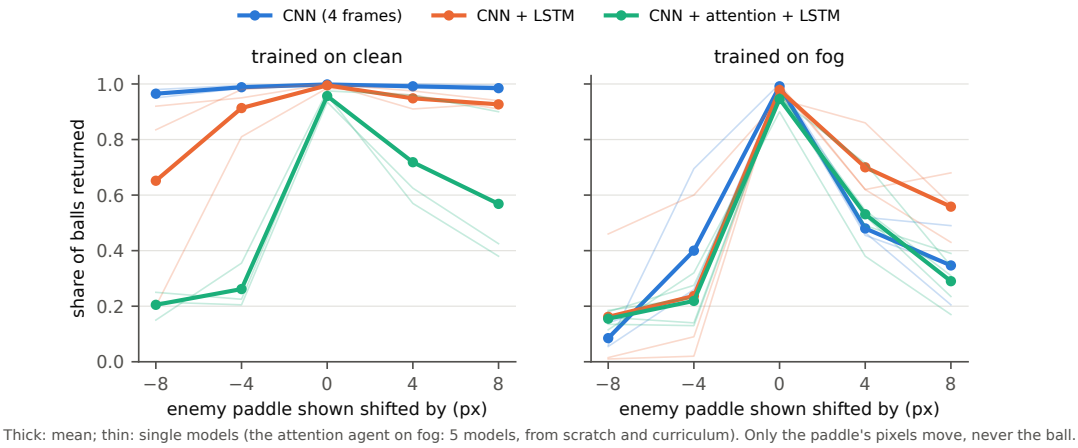


Figure 3: Share of balls returned when the opponent's paddle the model sees is moved by −8 to +8 px. Only the paddle's pixels move; the ball and the game are unchanged. Left: models trained on the clean view. Right: models trained on fog.

takes off, so 5M steps could not have been enough. Clean pre-training is therefore a good start for every agent. For the attention agent it is the difference between failing and winning.

Every model is a specialist. A fog + hidden paddle model loses about −20 on every other view, including the clean one: a ball visible in its own half is out of distribution for it.

3.2 Under fog, every agent plays through the opponent's paddle

If the fog really forced the agents to remember the ball, a fog-trained agent would not need the opponent's paddle. It does.

- **Erasing or mirroring the paddle** makes every fog-trained model, of every architecture, lose almost every point (−20.1 to −20.9). This holds even though the real ball still reaches its paddle.
- **A small shift is enough** (Figure 3): moving the paddle the agent sees by 4–8 px, half a paddle's height, costs fog agents most of their returns.

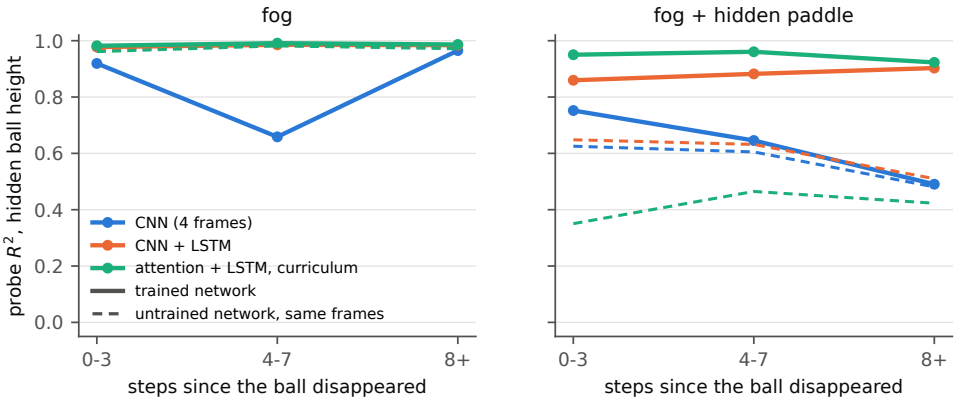


Figure 4: How well a linear probe reads the hidden ball's height from each model's internal state (LSTM $[h, c]$; for the CNN, its 512 features), by how many steps the ball has been in the agent's half. Solid: the trained model. Dashed: an untrained network of the same architecture fed the same frames.

- **Clean models differ by architecture.** The clean CNN does not notice the shift or the erasure, so it ignores the opponent's paddle. The clean LSTM partly relies on it. The clean attention agent depends on it almost entirely: its heads put 10–15 times the uniform weight on that paddle's cell, and it collapses when the paddle is hidden.

The agents do not simply copy the opponent's paddle height. When the paddle is shifted, 16 of the 17 fog agents that play move their own paddle the *opposite* way. In Pong, where the ball meets a paddle sets its return angle, so one reading is that the agents infer the ball's direction from where it met the opponent's paddle. This fits the sign of the effect but is not tested directly.

Scripted players put the shortcut in perspective. On the same games under fog:

- copying the opponent's paddle returns 55% of balls;
- going to the ball's last seen height returns 66%;
- extrapolating its seen path, an ideal short memory, returns 88% (96% without fog).

The shortcut is strong but not enough on its own. The fog + hidden paddle LSTM (98%) beats ideal extrapolation, and the CNN (88%) matches it.

3.3 With the shortcut gone, the memory agents remember the ball

Probes. Following Anand et al. [2] and Alain and Bengio [1], we fit ridge regressions, cross-validated over games, from each model's state to the ball's height while it crosses the agent's half. To separate memory from input, we also feed every frame to an untrained network of the same architecture, with its own memory, and probe it the same way (Figure 4).

- **Under fog alone, decodability is not memory.** Eight or more steps in, the hidden ball decodes almost perfectly from every model, including the memoryless CNN and the untrained network (R^2 0.92–0.99), because the visible opponent paddle tracks it.
- **On fog + hidden paddle, only trained memory holds the ball.** Eight or more steps into the fog, the LSTM's memory gives R^2 0.87–0.97 and the curriculum attention agent's 0.89–0.98, against 0.27–0.58 for the untrained network.

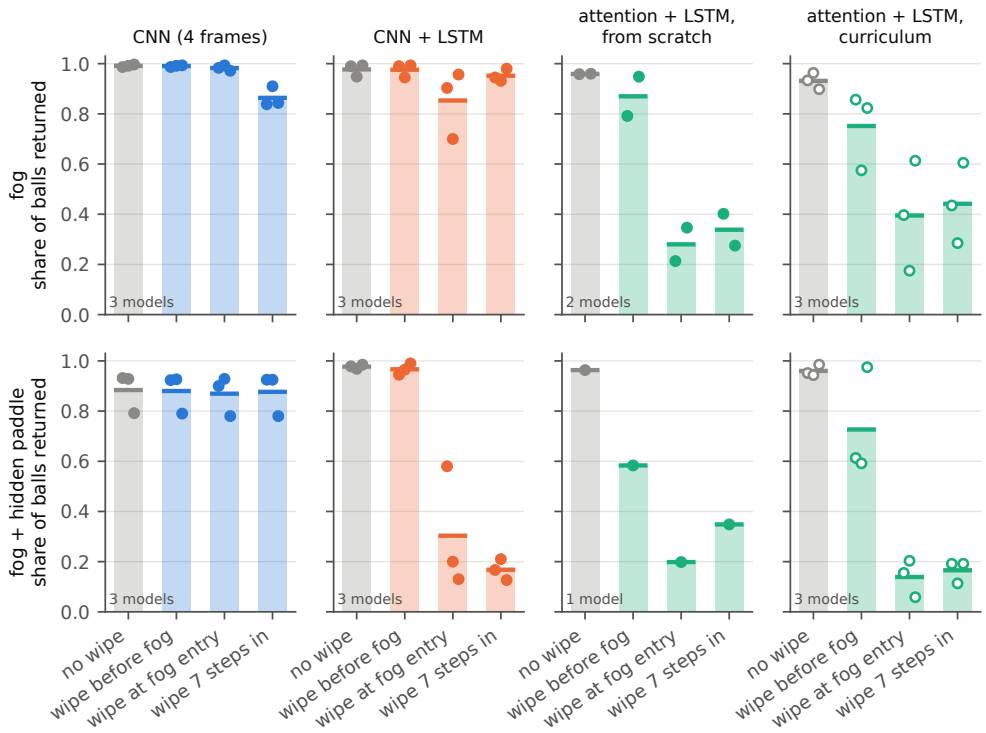


Figure 5: Share of balls returned when the memory is wiped (LSTM state set to zero; for the CNN, the frame stack filled with the current frame for four steps) before the fog, as the ball enters it, or seven steps in. Rows: training view. Bars: mean over models; dots: models.

Memory wipes. A probe shows that information is present, not that the agent uses it. We therefore wipe the memory at three moments and replay each ball (Figure 5).

- **The fog + hidden paddle LSTM depends on its memory.** A wipe as the ball enters the fog drops the share of balls returned from 0.98 to 0.30, and a wipe seven steps in drops it to 0.17. A wipe before the fog costs nothing, because the ball is seen again.
- **The fog-only LSTM barely needs memory** (0.85 after a wipe at fog entry): it rebuilds its estimate from the opponent's paddle.
- **The attention agent depends on its memory most of all.** On fog + hidden paddle, a wipe at fog entry leaves 0.06–0.20 of balls returned. Even on fog, with the opponent's paddle in view, it recovers poorly (0.17–0.61).

Unlike the LSTM agent, however, the attention agent is also hurt by a wipe *before* the fog (0.57–0.97 of balls returned). Its memory sets its attention query, so a wipe both erases what it remembers and disturbs where it looks. The attention block below separates the two.

Timescale. The forget gates of the fog + hidden paddle LSTMs, recomputed on their own games, give their ten slowest units half-lives of 18.5, 16.5 and 8.3 steps. These are about as long as the 14 steps the ball stays hidden. Clean LSTMs instead carry units that hardly forget at all.

3.4 The CNN without memory commits early

The CNN wins fog + hidden paddle with two seeds although no wipe of its frame stack changes anything. Its behaviour explains how. Four steps into the fog, its paddle is already 6–10 px from where the ball will arrive, against 13–18 px for the other agents, and it barely moves afterwards. It moves while the ball is visible, and its own paddle, which it always sees, carries the information

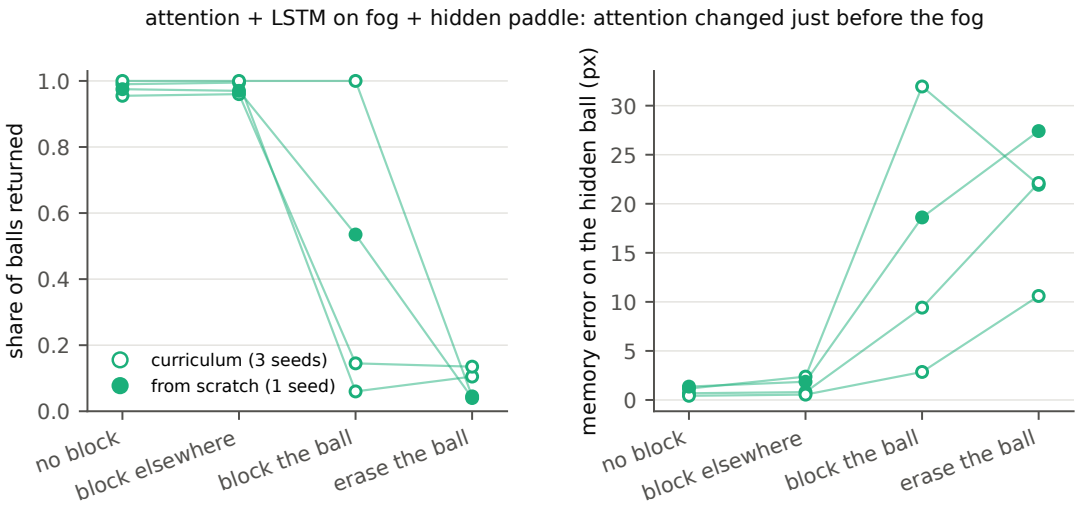


Figure 6: The attention agent on fog + hidden paddle. Each incoming ball is replayed while, as long as the ball is visible before the fog, the agent may not attend to the 3×3 cells around it (*block the ball*). The control blocks the same number of cells at the mirrored height (*block elsewhere*), and *erase the ball* removes the ball from the screen. Left: share of balls returned. Right: error of the memory probe on the hidden ball's height. Lines: models.

through the fog. This also explains why a probe reads some of the hidden ball from its features (R^2 0.40–0.57) as well as from an untrained network's: the information is in its paddle, an input, not in a memory.

3.5 The attention agent remembers only what it looked at

The attention agent's readout is the only path from the frame to its memory, so we can ask directly whether it remembers only what it attends to. While the ball is visible before the fog, we remove the cells around it from every head's softmax. Neither paddle's cells are ever blocked. Figure 6 shows the result for the four models that play this view.

- **Blocking the ball makes the agent forget it.** In three of four models, the probe's error on the hidden ball rises from about 1 px to 9–32 px, and the share of balls returned falls from 0.95–0.99 to 0.06–0.54.
- **The control has no effect.** Blocking the same number of cells elsewhere changes nothing (0.96–1.00), so the effect is specific to the ball.
- **In two models, attention is the whole gate.** The block does as much damage as erasing the ball from the screen.
- **One model is unaffected by the block but not by erasure**, plausibly because neighbouring cells, whose receptive fields overlap the ball, still let it through.

By contrast, the agents' natural attention to the ball does not predict their memory error flight by flight (Spearman -0.12 to $+0.50$). Only the intervention is informative.

3.6 Is the attention agent just too small?

The attention agent is 7–9 times smaller than the other two, so its failure from scratch could be a matter of size. We test this with a wider attention agent of 1.98M parameters, within 1.1% of the LSTM agent. It keeps the same 128-unit memory but has eight heads, ReLU values of 320 units and a 512-unit ReLU layer after the readout. We train it with exactly the protocol above:

from scratch on clean and on fog + hidden paddle, and the 10M + 5M curriculum, with three seeds each. [Results pending: the runs are training. In training, one of three from-scratch seeds already wins fog + hidden paddle at 6M steps. Report the frozen evaluation and whether size alone explains the failure.]

4 Related work

Deep Q-networks learned Atari games, Pong included, from pixels with four stacked frames [12, 13]. Hausknecht and Stone [8] replaced the stack with an LSTM and showed that recurrence helps when frames are randomly blanked (flickering Pong). Our occlusion is structured instead: one region, always hidden, chosen so that memory, a shortcut, or an early commitment are all possible solutions. Attention-augmented agents [14] let an LSTM query a spatial feature map, which makes where the agent looks visible; we use a simplified version and intervene on that attention rather than only inspecting it. Linear probes for Atari state follow AtariARI [2]. Shortcut learning [7] was diagnosed for Pong by SCoBots [6] and HackAtari [5]. We show that the shortcut persists under fog and how it is read. Clean-then-hard training is a simple curriculum [4].

5 Limitations

- **Scale.** We use three seeds per condition and a single game. Several findings rest on one or two models, such as the attention agent from scratch and the fog-only attention agents.
- **Size confound.** The attention agent differs from the others in size as well as in attention; Section 3.6 addresses this.
- **Evaluation.** It uses the stochastic policy, against the built-in opponent only.
- **Mechanism.** Reading the shortcut as an inference from where the ball met the opponent's paddle fits our data but is not tested directly.
- **Training noise.** The 15M-step fine-tuning controls were partly resumed after power cuts with a fresh optimiser; this is recorded in their checkpoints.

6 Conclusion

Agents learn to remember the ball only when nothing else will do. With the opponent's paddle in view, even a fog over their own half leaves every architecture playing through the shortcut. Without it, the LSTM agent carries the hidden ball in its memory and acts on it. The attention agent does too, but its memory holds only what its attention let in. The memoryless CNN wins without remembering by committing before the ball disappears. Each architecture used its own tools, and the one with the most structured tools needed the easier game first to learn to use them.

References

[1] Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes. *arXiv preprint arXiv:1610.01644*, 2016.

[2] Ankesh Anand, Evan Racah, Sherjil Ozair, Yoshua Bengio, Marc-Alexandre Côté, and R. Devon Hjelm. Unsupervised state representation learning in Atari. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[3] Marc G. Bellemare, Yavar Naddaf, Joel Veness, and Michael Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47:253–279, 2013.

[4] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th International Conference on Machine Learning*, pages 41–48, 2009.

[5] Quentin Delfosse, Jannis Blüml, Bjarne Gregori, and Kristian Kersting. HackAtari: Atari learning environments for robust and continual reinforcement learning. *arXiv preprint arXiv:2406.03997*, 2024.

[6] Quentin Delfosse, Sebastian Sztwiertnia, Mark Rothermel, Wolfgang Stammer, and Kristian Kersting. Interpretable concept bottlenecks to align reinforcement learning agents. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

[7] Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673, 2020.

[8] Matthew Hausknecht and Peter Stone. Deep recurrent Q-learning for partially observable MDPs. In *AAAI Fall Symposium on Sequential Decision Making for Intelligent Agents*, 2015.

[9] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9 (8):1735–1780, 1997.

[10] Shengyi Huang, Rousslan Fernand Julien Dossa, Chang Ye, Jeff Braga, Dipam Chakraborty, Kinal Mehta, and João G. M. Araújo. CleanRL: High-quality single-file implementations of deep reinforcement learning algorithms. *Journal of Machine Learning Research*, 23(274):1–18, 2022.

[11] Marlos C. Machado, Marc G. Bellemare, Erik Talvitie, Joel Veness, Matthew Hausknecht, and Michael Bowling. Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents. *Journal of Artificial Intelligence Research*, 61:523–562, 2018.

[12] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing Atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.

[13] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

[14] Alexander Mott, Daniel Zoran, Mike Chrzanowski, Daan Wierstra, and Danilo J. Rezende. Towards interpretable reinforcement learning using attention augmented agents. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[15] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.